SPARQL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

SPARQL

https://github.com/mikel-egana-aranguren/DBA

CC BY

SPARQL

SPARQL **P**rotocol and **R**DF **Q**uery **L**anguage

SPARQL Endpoint publikoak

Wikidata Query Service

DBpedia SPARQL

UniProt SPARQL

Open Data Euskadi

datos.gob.es

SPARQL ikasteko baliabideak

Learning SPARQL (liburua)

Wikidata SPARQL Tutorial

Wikidata Query Service (adibideekin)

SPARQL W3C (=HTML, !=SQL)

SPARQL 1.1 Query Language: SPARQL kontsultak RDF grafoetan

SPARQL 1.1 Update: RDF grafoak aldatu SPARQL bidez

SPARQL 1.1 Protocol: SPARQL kontsultak HTTP bidez

SPARQL 1.1 Graph Store HTTP Protocol: RDF grafoak HTTP bidez aldatu

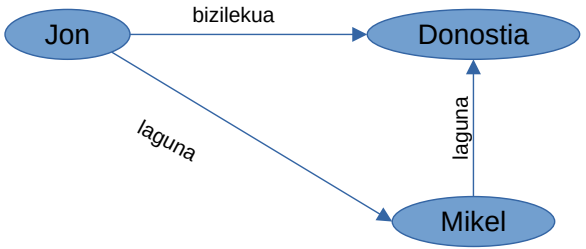
SPARQL 1.1 Service Description: SPARQL endpoint-ak RDF bidez deskribatu

SPARQL 1.1 Federated Query: SPARQL endpoint-ak konbinatu

GraphDB SPARQL 1.1 compliance (~~SPARQL 1.1 Service Description~~)

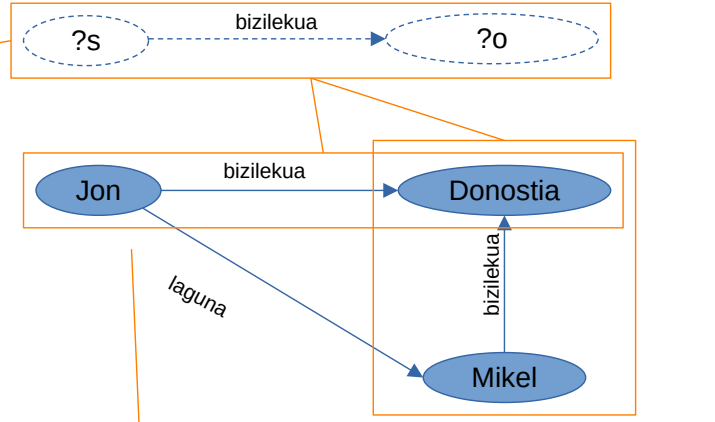
SPARQL

```
SELECt ?s ?o
WHERE {
      ?s bizilekua ?o
}
```

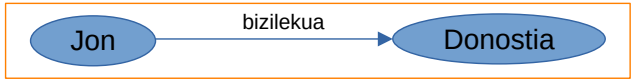


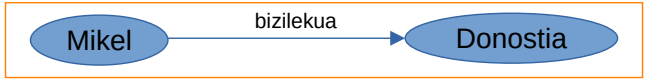
SPARQL

```
SELECt ?s ?o
WHERE {
      ?s bizilekua ?o
}
```



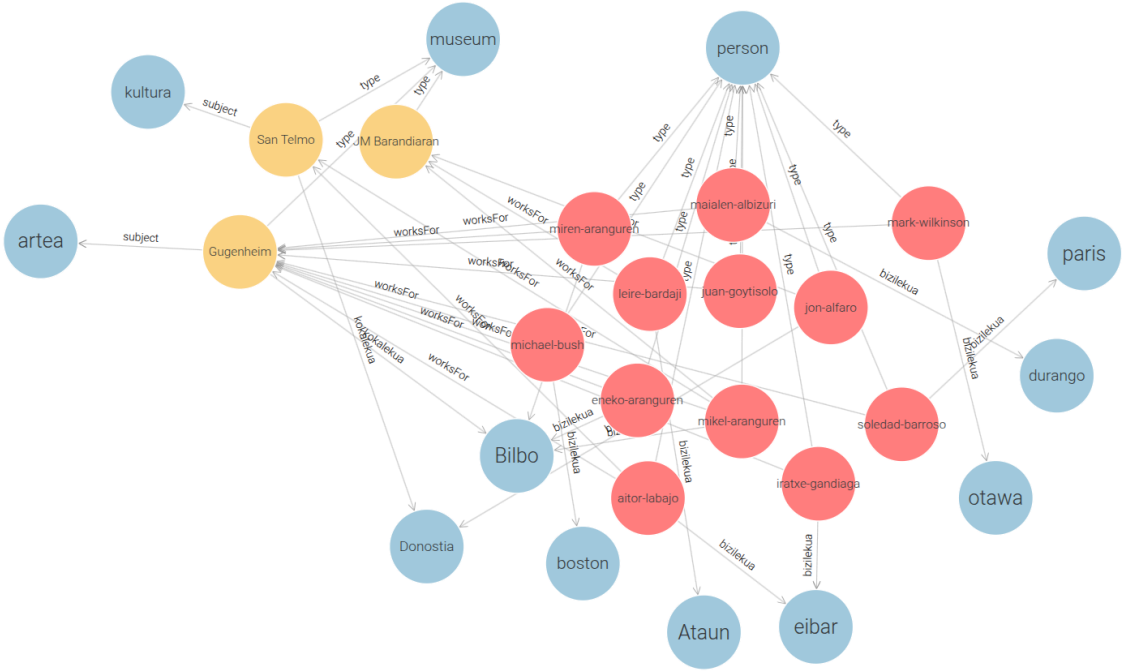
?s	?o
Jon	Donostia
Mikel	Donostia





Museoen dataset-a

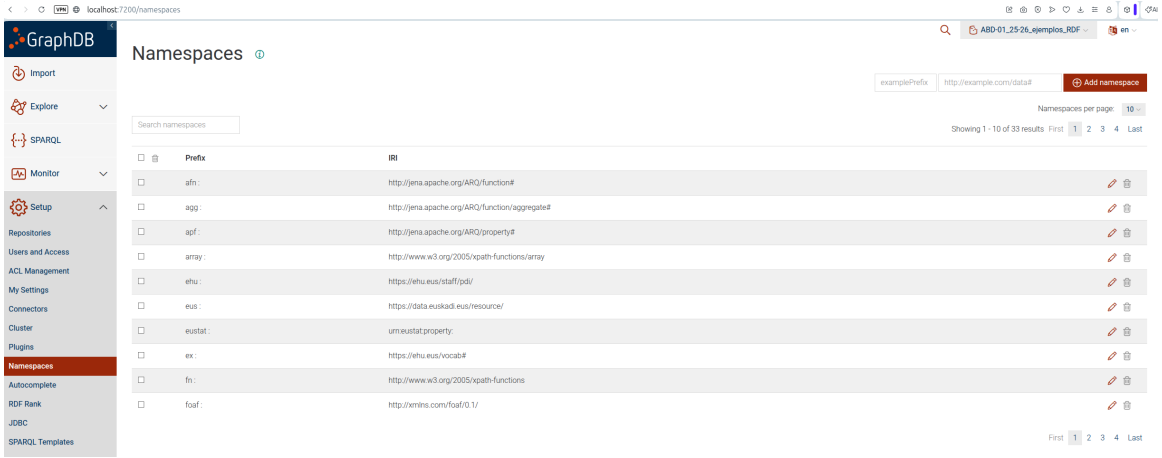
[Museoak.rdf]



Museoen dataset-a

```
# Prefix-ak
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>
PREFIX dbpedia: <http://dbpedia.org/ontology/>
PREFIX gov: <http://vocab.data.gov/def/drm#>
```

GraphDB namespace-ak



Kontsultaren egitura

```
# Jaso nahi ditugun aldagaiak (Denak = "*")
SELECT ?subjektua ?klasea

# Grafo nagusitik atera nahi dugun grafo-patroia
WHERE {
  ?subjektua gip_prop:bizilekua gip:donostia .
  ?subjektua rdf:type ?klasea
}
```

OPTIONAL

```
SELECT ?langilea ?bizilekua

WHERE {
  # Pertsona izan behar du
  ?langilea rdf:type foaf:person .

  # Nonbaiten bizi behar du
  ?langilea gip_prop:bizilekua ?bizilekua
}
```

OPTIONAL

```
SELECT ?langilea ?bizilekua

WHERE {
  # Pertsona izan behar du
  ?langilea rdf:type foaf:person .

  # Leku batean bizi daiteke edo ez
  OPTIONAL {
    ?langilea gip_prop:bizilekua ?bizilekua
  } }
```

UNION

```
SELECT ?entitatea ?izena

WHERE {
  { ?entitatea rdfs:label ?izena }
  UNION
  { ?entitatea foaf:name ?izena }
}
```

Emaitzak ordenatu

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena) # DESC edo ASC izan daiteke
```

Emaitzak mugatu

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena)
LIMIT 3
```

Orrialdekatzea

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena)
LIMIT 3
OFFSET 3 # OFFSET 6
```

Emaitzak filtratu

```
SELECT ?museoa ?langile_kop

WHERE {
  ?museoa gip_prop:kopurua ?langile_kop .
  FILTER (?langile_kop > "800"^^xsd:int)
}
```

Emaitzak filtratu

```
SELECT ?langilea ?izena

WHERE {
  ?langilea foaf:name ?izena .
  FILTER regex(?izena,'Mi.*')
}
```

Emaitzak filtratu

Logika: !, &&, ||

Kalkuluak: +, -, *, /

Alderapenak: =, !=, >,<

Emaitzak filtratu

SPARQL test-ak: isURI, isBlank, isLiteral, bound

Datuetan sartu: str, lang, datatype

Gehiago: sameTerm, langMatches, regex, …

Duplikatuak ekidin

```
SELECT ?lantokia # DISTINCT
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?lantokia
}
```

ASK

```
ASK
WHERE {
  ?person gov:worksFor gip:gugenheim
}
```

DESCRIBE

```
DESCRIBE <http://gipuzkoa.eus/resource/mikel-aranguren>
```

[Visual]

GRAFOAK

Grafoa: tripleen multzoa

Multzo osoa URI batekin identifikatzen da (datuena ez dena)

Triple Store guztiek Default Graph bat dute

GRAFOAK

```
INSERT DATA{
  GRAPH <http://example/bookStore> {
    <http://example/book1> gip_prop:price 42
  }
}
```

```
SELECT *
FROM <http://example/bookStore> # Grafo URI gabe probatu
WHERE {
  ?s ?p ?o .
}
```

Agregazio funtzioak

COUNT: elementuak zenbatu

SUM: balioen batura

AVG: batez bestekoa

Agregazio funtzioak

MIN / MAX: minimoa eta maximoa

GROUP_CONCAT: balioak kateatu

SAMPLE: adibide-balio bat lortu

COUNT

```
SELECT (COUNT(?person) AS ?guztira)
WHERE {
  ?person rdf:type foaf:person
}
```

GROUP BY

Emaitzak aldagai baten balioaren arabera multzokatu

```
SELECT ?hiria (COUNT(?museo) AS ?museoKop)
WHERE {
  GRAPH <urn:count> {
    ?museo rdf:type dbpedia:museum ;
    gip_prop:kokalekua ?hiria .
  }
}
GROUP BY ?hiria
ORDER BY DESC(?museoKop)
```

[Museoak-count.ttl]

HAVING

Taldeak filtratu agregazioaren ondoren

```
SELECT ?hiria (COUNT(?museo) AS ?museoKop)
WHERE {
  GRAPH <urn:count> {
    ?museo rdf:type dbpedia:museum ;
    gip_prop:kokalekua ?hiria .
  }
}
GROUP BY ?hiria
HAVING (COUNT(?museo) > 1)
ORDER BY DESC(?museoKop)
```

BIND

Balioak aldagaiei esleitu kontsultaren barruan

```
SELECT ?person ?fullName
WHERE {
  GRAPH <urn:name-surname>{
    ?person foaf:firstName ?first ;
    foaf:lastName ?last .
  }
BIND(CONCAT(?first, " ", ?last) AS ?fullName)
}
```

[Museoak-name-surname.ttl]

Kate-funtzioak

CONCAT: kateak batu

STRLEN: katearen luzera

SUBSTR: azpikatea

UCASE / LCASE: maiuskulak / minuskulak

Kate-funtzioak

STRSTARTS / STRENDS / CONTAINS: kateetan bilatu

REPLACE: testua ordeztu

REGEX: adierazpen erregularrak

REGEX

```
SELECT ?museo ?izena
WHERE {
  ?museo rdf:type dbpedia:museum ;
    rdfs:label ?izena .
  FILTER REGEX(?izena, "Gugen")
}
```

Funtzio numerikoak

ABS: balio absolutua

ROUND / CEIL / FLOOR: biribiltzea

RAND: ausazko zenbakia

Eragileak: +, -, *, /

Data/ordu funtzioak

NOW(): uneko data eta ordua

YEAR / MONTH / DAY: osagaiak atera

HOURS / MINUTES / SECONDS: ordu zehatza

VALUES

Inline balioak aldagaiei eman (Kontsulta parametrizatuetarako baliagarria)

```
SELECT ?museo ?obrak
WHERE {
  VALUES ?museo {
    <http://gipuzkoa.eus/resource/gugenheim>
    <http://gipuzkoa.eus/resource/san-telmo>
  }
  ?museo gip_prop:kopurua ?obrak
}
```

MINUS

Patroi batekin bat datozen emaitzak baztertu: museo guztiak, 300 obra baino gehiago dituzten museoak kenduta

```
SELECT ?museo ?obrak
WHERE {
  ?museo rdf:type dbpedia:museum .
  MINUS {
    ?museo gip_prop:kopurua ?obrak .
    FILTER (?obrak > 300)
  }
}
```

NOT EXISTS

Patroi baten existentzia ezaren arabera filtratu

EXISTS vs MINUS

EXISTS/NOT EXISTS: filtroa, ez ditu binding-ak ezabatzen

MINUS: soluzio osoak ezabatzen ditu

NOT EXISTS eraginkorragoa da balioztatzerako

MINUS erabilgarriagoa da patroi konplexuen bazterketarako

Property Paths

^: alderantzizkoa (ex:parent^ seme-alabak aurkitzen ditu)

: zero edo gehiago (ex:knows ezagunen sarea)

+: bat edo gehiago (ex:subClassOf+ hierarkiarako)

!: edozein erlazio hori izan ezik

Property Paths

?: aukerakoa (zero edo bat)

|: alternatiboa (ex:email|ex:phone)

/: sekuentzia (ex:livesIn/ex:country)

Property paths

```
SELECT ?person ?city
WHERE {
  ?person gov:worksFor/gip_prop:kokalekua ?city . # deskonposatu
}
```

SPARQL 1.1 Update

W3C-ren estandarra (=HTML, !=SQL)

DELETE DATA

```
DELETE DATA {
  <http://gipuzkoa.eus/resource/aitor-labajo>
  rdf:type gip:hiritar
}
# DESCRIBE <http://gipuzkoa.eus/resource/aitor-labajo>
```

DELETE WHERE

```
DELETE {?person rdf:type gip:hiritar}
WHERE {?person foaf:name ?name}

#SELECT * WHERE {
#  ?person rdf:type gip:hiritar
#}
```

INSERT DATA

```
INSERT DATA {
  gip:aitor-labajo rdf:type foaf:person
}
# DESCRIBE gip:aitor-labajo
```

INSERT WHERE

```
INSERT {
  gip:jon-alfaro rdf:type ?type .
}
WHERE {
  gip:aitor-labajo rdf:type ?type .
}
# DESCRIBE <http://gipuzkoa.eus/resource/jon-alfaro>
```

DELETE/INSERT konbinatua (UPSERT)

Email-a modu transakzionalean eguneratu:

```
DELETE { ?person ex:email ?oldEmail }
INSERT { ?person ex:email ?newEmail }
WHERE {
  ?person foaf:name "Mikel" ;
    ex:email ?oldEmail .
  BIND("mikel@new.com" AS ?newEmail)
}
```

CONSTRUCT

```
CONSTRUCT {
  ?langilea rdf:type gip:gugenheim_langilea
}
WHERE {
  ?langilea gov:worksFor gip:gugenheim
}
```

LOAD

(=HTML, !=SQL)

```
LOAD <https://rest.uniprot.org/uniprotkb/P01308.rdf>
INTO GRAPH <urn:uniprot>
```

CLEAR

```
CLEAR GRAPH <urn:uniprot>

# CLEAR DEFAULT

# CLEAR ALL
```

DROP

```
DROP GRAPH <http://example.org/myGraph>

# DROP ALL
```

COPY / MOVE / ADD

COPY: grafo bat beste batera kopiatu (helburua gainidazten du)

MOVE: grafoa mugitu (kopiatu + jatorrizkoa ezabatu)

ADD: tripleak helburura gehitu (gainidatzi gabe)

COPY / MOVE / ADD

```
COPY <urn:name-surname>
TO <urn:nombre-apellidos>

COPY DEFAULT
TO <urn:backup>
```

SPARQL 1.1 Protocol

W3C-ren estandarra (=HTML, !=SQL)

SPARQL 1.1 Protocol

HTTP GET query parametroarekin: `GET /sparql?query=SELECT...`

HTTP POST (kontsulta luzeagoetarako): `POST /sparql` body URL-encoded-ekin

Content negotiation (Accept header): application/sparql-results+json, text/csv, application/rdf+xml, text/turtle

[adibideak/post.sh]

SPARQL 1.1 Protocol

Python: RDFLib, SPARQLWrapper

Java: Apache Jena, RDF4J

SPARQL 1.1 Service Description

W3C-ren estandarra (=HTML, !=SQL)

SPARQL 1.1 Service Description

SPARQL endpoint-ei buruzko metadatuak eskaintzen ditu bezeroei laguntzeko:

- **Gaitasunak aurkitu**: Zer funtzionalitate onartzen ditu? (agregazioak, property path-ak, ezeztapena, etab.)
- **Dataset eskuragarriak aurkitu**: Zer Named Graph daude eskuragarri?
- **Formatu bateragarriak zehaztu**: JSON, XML, CSV, ...
- **Inferentzia-erregimenak ikusi**: RDFS, OWL, ...

SPARQL 1.1 Service Description

```
curl -H "Accept: text/turtle"
https://dbpedia.org/sparql/
```

[adibideak/dbpedia_service_description.ttl]

SPARQL 1.1 Federated Query

W3C-ren estandarra (=HTML, !=SQL)

Kontsulta federatuak

Hainbat SPARQL endpoint kontsultatzea ahalbidetzen dute kontsulta bakarrean

Jatorri desberdinetako datuak konbinatu `SERVICE` hitz gakoa erabiliz

Kontsulta federatuak

```
INSERT DATA {
  <http://gipuzkoa.eus/resource/bilbo> owl:sameAs <http://dbpedia.org/resource/Bilbao> .
}
```

```
SELECT ?pop
WHERE {
  gip:bilbo owl:sameAs ?town .
  SERVICE <https://dbpedia.org/sparql/> {
    ?town <http://dbpedia.org/ontology/populationTotal> ?pop .
  }
}
```

SPARQL 1.1 Graph Store HTTP Protocol

W3C-ren estandarra (=HTML, !=SQL)

Eragiketak

RDF grafoak kudeatu HTTP bidez:

- **GET**: Grafoa berreskuratu
- **PUT**: Grafoa ordeztu (Suntsitzailea)
- **DELETE**: Grafoa ezabatu
- **POST**: Tripleak grafoari gehitu

Eragiketak: Parametroak

Oinarrizko endpoint-a: konfigurazioaren arabera

Parametroak:

- `graph=<URI>` : Named Graph zehaztu
- `graph` gabe: Default Graph-ean du eragina

Content-Type: RDF/XML, Turtle, N-Triples, etab.

[adibideak/graph_store_protocol.sh]

Abantailak

- **RESTful**: HTTP estandarrean oinarritutako interfaze sinplea
- **Automatizazioa**: Script eta CLI tresnekin integratzea
- **SPARQL UPDATE gabe eguneratzeak**: INSERT/DELETE-ren alternatiba
- **Bertsioen kudeaketa**: Grafoen bertsio osoak ordeztu

Graph Store vs SPARQL UPDATE

Alderdia	Graph Store HTTP	SPARQL UPDATE
Granularitatea	Grafo osoa	Triple zehatzak
Interfazea	REST/HTTP	SPARQL kontsulta
Ohiko erabilera	Bertsio osoak ordeztu	Aldaketa zehatzak
Formatua	RDF (TTL, RDF/XML, etab.)	SPARQL INSERT/DELETE

GraphDB Ontop: Datu erlazionalen birtualizazioa

Ontop SQL datu-base erlazionalak SPARQL bidez kontsultatzeko GraphDB plugin-a

Datuak **ez dira kopiatzen** triplestore-ra: denbora errealean eskuratzen dira (birtualizazioa)

Ontop-ek SPARQL kontsultak SQLra automatikoki itzultzen ditu: erabiltzaileak SPARQL erabiltzen du datuak SQLn daudela jakin gabe

GraphDB Ontop: Arkitektura

- **Datu-base erlazionala**: dagoen SQL datu-iturria
- **Ontologia**: RDF hiztegia definitzen du (klaseak eta propietateak)
- **R2RML mapping-a**: SQL taula/zutabeen eta RDF grafoaren arteko mapeoa
- **Ontop/GraphDB**: SPARQL → SQL denbora errealean itzultzen duen motorea

R2RML: RDB to RDF Mapping Language

R2RML W3C estandar bat da, datu-base erlazionaletatik RDFra mapeoak definitzeko

- **Triples Map**: SQL taula edo kontsulta batetik RDF tripleak nola sortu definitzen du
- **Logical Table**: jatorrizko SQL taula edo SQL kontsulta
- **Subject Map**: subjektuaren URIa nola sortu
- **Predicate-Object Map**: predikatuak eta objektuak nola sortu

R2RML: Mapeo adibidea

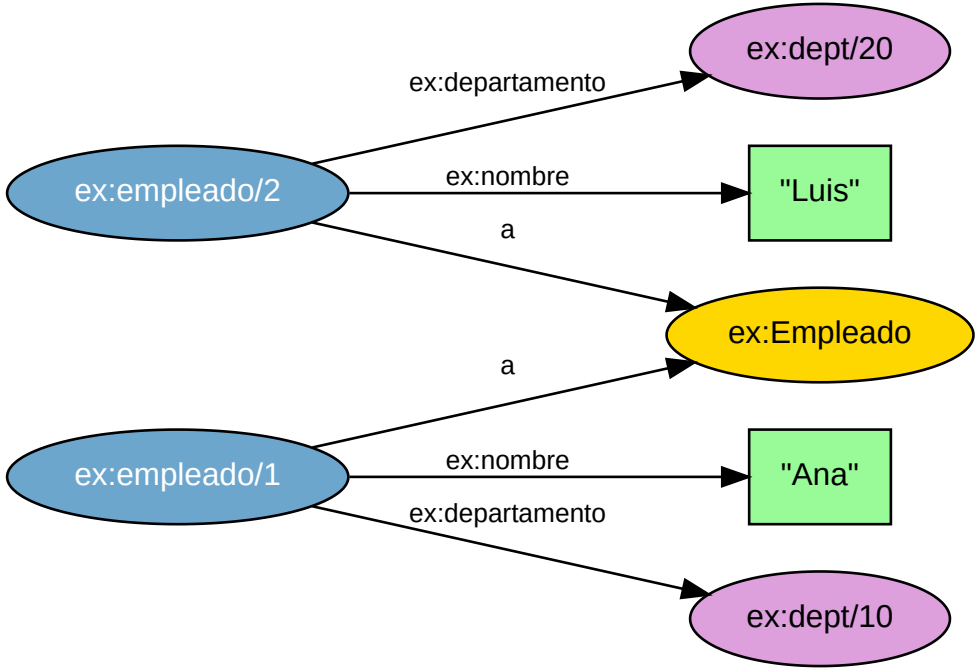
SQL taula: `empleados(id, nombre, dept_id)`

```
<#TriplesMap1> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "empleados" ] ;
  rr:subjectMap [
    rr:template "http://ejemplo.org/empleado/{id}" ;
    rr:class ex:Empleado
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:nombre ;
    rr:objectMap [ rr:column "nombre" ]
  ] ;
    rr:predicateObjectMap [
      rr:predicate ex:departamento ;
      rr:objectMap [
        rr:template "http://ejemplo.org/dept/{dept_id}" ]
  ] .
```

R2RML: Mapeoaren emaitza

id	nombre	dept_id
1	Ana	10
2	Luis	20

R2RML: Mapeoaren emaitza



GraphDB Ontop: Abantailak

- Datuak ez dira bikoizten: datuek datu-base erlazionalean jarraitzen dute
- Datuak beti eguneratuta (denbora errealeko sarbidea)
- W3C estandarra (R2RML): motorren artean eramangarria

GraphDB Ontop: Mugak

- Errendimendua datu-base erlazionalaren araberakoa da
- SPARQL funtzio guztiek ez dute SQLrako itzulpen zuzenik

GraphDB Explain Plan

GraphDB join flow: partekatutako aldagaiekin patroien arteko join-ak ebaluatu (SPARQL: inner join inplizitua)

GraphDB Explain Plan: ebaluazio ordena optimizatua aztertu

Kontsultari `FROM onto:explain` gehituta aktibatzen da

(`onto:measure` aukerak exekuzio-denborak ematen ditu)

GraphDB Explain Plan

```
PREFIX onto: <http://www.ontotext.com/>
SELECT * FROM onto:explain # onto:measure
WHERE {
  ?s ?p ?o.
}
```

```
SELECT ?s ?p ?o
{

  { # ----- Begin optimization group 1 -----

    ?s ?p ?o . #    Collection size: 157    Predicate collection size: 157  Unique subjects: 144    Unique objects: 87  Current complexity: 157

  } # ----- End optimization group 1 -----
  # ESTIMATED NUMBER OF ITERATIONS: 157

}

# NOTE: Optimization groups are evaluated one after another exactly in the given order.
# If there are too many optimization groups consisting of a single statement pattern,
# then one should try to relocate the following clauses by hand:
# VALUES, BIND, OPTIONAL, property paths with '*' and/or '+' (the latter can be also surrounded with brackets).
# Sub-SELECTs will always be evaluated first.
```

GraphDB Explain Plan

Optimization group: tripleen taldea, optimizazioaren arabera ordenatua

Collection size: patroiarekin bat datozen triple kopurua

Predicate collection size: predikatu hori duten errepositorioko triple guztien kopurua

Unique subjects: patroiarekin bat datozen subjektu bakarren kopurua

GraphDB Explain Plan

Unique objects: patroiarekin bat datozen objektu bakarren kopurua

Current complexity: datu-baseak patroi horretarako egin beharko dituen bilaketa atomikoen konplexutasuna

Estimated number of iterations: talde honetarako exekutatuko diren iterazioak

GraphDB Explain Plan

Talde barruko tripleen ordena selektibitatearen arabera

Patroi baten selektibitatea: collection size (predikatuaren karga), join-ak (subjektu/objektu bakarrak), eta aurreko patroian ebatzitako aldagaiak

Explain Plan

```
PREFIX onto: <http://www.ontotext.com/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX gov: <http://vocab.data.gov/def/drm#>

SELECT * FROM onto:explain
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
  ?person foaf:name ?nombre .
}
```

```
[INSERT {?person rdf:type foaf:person . } WHERE { ?person gov:worksFor ?museo . ?person foaf:name ?nombre . }]
```

Explain Plan: Patroien ordena

foaf:name: collection size 13 (txikiena) eta predicate collection size ere 13:

foaf:name triple horietan bakarrik agertzen da, oso selektiboa

gov:worksFor collection size 16, baina ?person dagoeneko ebatzita dago

rdf:type collection size 54, 45 subjektu bakar eta objektu bakarrekin

Konplexutasuna = collection size / selektibitatea (ebatzitako aldagaiak barne)

GraphDB Explain Plan

Talde barruko tripleak: automatikoki optimizatzen dira

Taldeen ordena: ez da automatikoki optimizatzen, kontsultan agertzen diren ordenan ebaluatzen dira (erabiltzaileak optimizatu behar du)

Taldeak join flow-a eteten denean agertzen dira

Join flow-a eteten dute

OPTIONAL: left join osoa lehenik ebatzi behar da eta gero aukerako zatia

UNION: adar bakoitzaebaluatu eta ondoren konbinatu

sub-SELECT: beti lehenengo ebaluatzen da

FILTER, BIND, VALUES: aurreko aldagaien menpe daude

Join flow-a eteten dute

* edo + duten property paths: hedapen errekurtsiboa

Aldagai partekaturik gabeko patroi deskonektatuak

Explain Plan: FILTER goiztiarra vs berantiarra

A kontsulta: FILTER amaieran (eraginkortasun txikia)

```
SELECT * FROM onto:explain
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
  ?person foaf:name ?nombre .
  FILTER regex(?nombre, "Mi.*")
}
```

Lehenik konbinazio GUZTIAK sortzen ditu, eta ondoren iragazten du

Explain Plan: FILTER goiztiarra vs berantiarra

B kontsulta: FILTER ?nombre-ren ondoren berehala (eraginkorra)

```
SELECT * FROM onto:explain
WHERE {
  ?person foaf:name ?nombre .
  FILTER regex(?nombre, "Mi.*")
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
}
```

Goiz iragazi → tarteko emaitza gutxiago → join gutxiago

Explain Plan: OPTIONAL-en eragina

```
SELECT * FROM onto:explain
WHERE {
  ?museo rdf:type dbpedia:museum .
  ?museo rdfs:label ?nombre .
  OPTIONAL {
    ?museo gip_prop:kopurua ?obras .
  }
  OPTIONAL {
    ?museo gip_prop:kokalekua ?ciudad .
  }
}
```

OPTIONAL bakoitzak Left Join bat gehitzen du → kostu handiagoa

Explain Plan: Beharrezkoa ez den OPTIONAL saihestu

Badakigunean museo guztiek kokalekua eta kopurua dutela:

```
SELECT * FROM onto:explain
WHERE {
  ?museo rdf:type dbpedia:museum .
  ?museo rdfs:label ?nombre .
  ?museo gip_prop:kopurua ?obras .
  ?museo gip_prop:kokalekua ?ciudad .
}
```

OPTIONAL gabe: Inner Join, eraginkorragoa